

Interview Questions & Answers — Experienced Candidate

A consolidated Q&A bank covering ML/AI, MLOps, Azure data & AI stack, Power BI/DAX, and supporting tools. Answers are pitched at 4+ years of hands-on experience — expect follow-ups on trade-offs, scaling, and production concerns.

1. Machine Learning & AI

Q1. Walk me through how you would approach a regression problem end-to-end. What pitfalls do you watch for?

Answer: I start by clarifying the business KPI and how the model's predictions will be consumed — this drives the choice of loss function and acceptable error. Then:

1. **EDA and target analysis** — distribution of the target, skew, outliers, presence of zero-inflation. For heavily skewed targets I consider a log/Box-Cox transform.
2. **Feature engineering** — domain features first (ratios, deltas, lags for time-series), then encoding strategies (target/one-hot/ordinal depending on cardinality), and interaction terms where domain logic supports them.
3. **Baseline** — a simple model (linear regression or median predictor) is essential. Without it you can't tell whether a gradient boosting model is actually adding value.
4. **Model family selection** — linear/regularized (Ridge, Lasso, ElasticNet) for high interpretability; tree ensembles (XGBoost, LightGBM, CatBoost) for tabular performance; deep models only when there's a justification.
5. **Validation strategy** — k-fold for IID data, time-series split for temporal data, group k-fold when there's leakage risk via entity ID.
6. **Metric choice** — RMSE penalizes outliers heavily; MAE is more robust; MAPE breaks near zero. I pick the one aligned with business cost.

Pitfalls: target leakage from features computed using future information, encoding the test set with training-derived statistics, ignoring heteroscedasticity, and validating on the wrong split (random split on time-series is the classic mistake).

Q2. Explain the bias-variance tradeoff and how it manifests in regularization.

Answer: Bias is error from oversimplified assumptions — a linear model trying to fit a curved relationship will be biased. Variance is sensitivity to the training sample — a deep tree memorizes noise.

L2 regularization (Ridge) shrinks coefficients toward zero, reducing variance at the cost of a small bias increase. L1 (Lasso) does the same plus performs feature selection by zeroing out weights, which helps when many features are irrelevant. ElasticNet combines both and is my default when I'm uncertain. In tree ensembles, the analogues are `max_depth`, `min_samples_leaf`, `subsample`, and `colsample_bytree` — each pushes the model toward higher bias but lower variance. The sweet spot is found via cross-validated hyperparameter search.

Q3. How do you choose between K-Means, DBSCAN, and hierarchical clustering?

Answer: It depends on the shape of the data and the question being asked.

- **K-Means** — fast, scales well, but assumes spherical clusters of similar size and requires `k` upfront. I use the elbow method or silhouette score to pick `k`. Sensitive to feature scaling, so I always standardize first.
- **DBSCAN** — great when clusters are arbitrarily shaped or you don't know `k`. It also identifies noise points natively, which is valuable for anomaly detection. The challenge is tuning `eps` and `min_samples`, and it struggles with varying cluster densities (HDBSCAN handles that better).
- **Hierarchical (agglomerative)** — useful when you want a dendrogram to inspect cluster relationships, or when the dataset is small. Doesn't scale beyond ~10k points without approximation.

For high-dimensional data I usually reduce dimensions first (UMAP or PCA) before clustering — distance metrics lose meaning in high dimensions.

Q4. What is feature engineering really, and how do you handle high-cardinality categorical features?

Answer: Feature engineering is encoding domain knowledge in a form a model can exploit. It's the highest-leverage activity in most tabular ML projects.

For high-cardinality categoricals (e.g., ZIP code, product SKU with 50k+ levels):

- **Target/mean encoding** with smoothing and out-of-fold computation to prevent leakage.
- **Frequency encoding** — replace category with its count; preserves information cheaply.
- **Hashing trick** for very high cardinality when you can tolerate collisions.
- **Embeddings** when feeding to a neural net — learnable representations that capture similarity between categories.
- **Grouping** — collapse rare levels into "other" to avoid overfitting on long-tail values.

CatBoost handles high cardinality well natively via ordered target statistics, which is one reason I default to it for messy tabular data.

Q5. Precision vs Recall — when do you optimize for which, and what about RMSE vs MAE?

Answer: Precision matters when false positives are costly — spam filtering, fraud alerts that trigger manual review. **Recall** matters when false negatives are costly — cancer screening, security threat detection. The F1 score balances them, but in practice I look at the PR curve and pick a threshold that matches the cost ratio.

For regression: **RMSE** squares errors so it punishes large mistakes more — appropriate when big misses are disproportionately bad (e.g., demand forecasting where a 100-unit miss is more than 10× worse than a 10-unit miss). **MAE** treats all errors linearly — better when the cost of error is roughly proportional. **MAPE** is interpretable as a percentage but explodes near zero and is asymmetric. For asymmetric costs I use quantile loss directly.

Q6. Explain RAG architecture and when it beats fine-tuning.

Answer: Retrieval-Augmented Generation injects relevant context at inference time. The pipeline:

1. **Indexing (offline)** — chunk documents, embed each chunk with an embedding model, store vectors + metadata in a vector DB.
2. **Retrieval (online)** — embed the user query, do approximate nearest-neighbor search, return top-k chunks.
3. **Generation** — pass the retrieved chunks plus the question into an LLM with a prompt that instructs it to answer grounded in that context.

RAG beats fine-tuning when:

- The knowledge changes frequently (product catalogs, policy docs).
- You need citations and traceability to source documents.
- The dataset is too small or noisy to fine-tune on.
- You want to update knowledge without retraining.

Fine-tuning wins when:

- You need to teach a *style, format, or reasoning pattern* rather than facts.
- Latency or token budget is tight (fewer tokens at inference).
- The domain language is so specialized that base model embeddings retrieve poorly.

In practice the two are complementary — fine-tune for behavior, retrieve for facts.

Q7. Walk me through how you'd build a production RAG system. What goes wrong?

Answer: Architecture: ingestion pipeline → chunking → embedding → vector store → retriever → reranker → LLM → response with citations. I'd add evaluation harness, monitoring, and a feedback loop.

Things that go wrong:

- **Bad chunking** — fixed-size chunks split sentences mid-thought. I use recursive or semantic chunking with overlap (typically 10–20%) and respect document structure (headers, sections).
 - **Embedding-query mismatch** — user questions and document chunks live in different "shapes." Hybrid search (BM25 + dense) or query rewriting helps. HyDE (hypothetical document embeddings) is also useful.
 - **Retrieval ≠ relevance** — top-k by cosine isn't always relevant. Add a cross-encoder reranker on top-50 to surface top-5.
 - **Lost-in-the-middle** — LLMs attend more to start/end of context. Place the most relevant chunk first.
 - **Context bloat** — stuffing 20 chunks degrades quality and costs tokens. Smaller, better-ranked context wins.
 - **No grounding enforcement** — without a strict prompt, the model hallucinates. I instruct it to answer "I don't know" if context is insufficient and to cite chunk IDs.
 - **Evaluation gap** — I use RAGAS or similar to track faithfulness, answer relevance, context precision, and context recall.
-

Q8. What is LangChain and where does it help vs. get in the way?

Answer: LangChain is an orchestration framework for LLM applications. It provides abstractions for chains, agents, memory, retrievers, and tool use.

Where it helps:

- Rapid prototyping — getting a RAG pipeline running in 50 lines.
- Standard integrations with vector stores, LLM providers, and document loaders.
- Reusable patterns (conversational retrieval chain, agent with tools).
- LangSmith for tracing and debugging.

Where it gets in the way:

- Abstractions leak — debugging chained calls through layers of wrappers is painful.
- API churn — versions break frequently.
- For production, the orchestration logic is often simple enough that raw API calls plus a thin custom wrapper are easier to maintain.

My pattern: prototype with LangChain (or LangGraph for stateful workflows), then evaluate whether to keep it or rewrite once the design solidifies. For agentic flows I prefer LangGraph because state transitions become explicit.

Q9. How do you do prompt engineering systematically for an experienced team?

Answer: Treat prompts as code: version-controlled, tested, evaluated.

Core techniques:

- **Role + task + constraints + format** structure. Be explicit about output format (JSON schema, markdown sections).
- **Few-shot examples** when the task is novel or format-sensitive. Pick examples that span the input distribution.
- **Chain-of-thought** for reasoning tasks — "think step by step" or structured scratchpads.
- **Decomposition** — break complex tasks into sub-prompts (extract → reason → format) rather than one mega-prompt.
- **Output guardrails** — schema validation, retry on parse failure, fallback prompts.
- **Negative instructions** sparingly; positive instructions are followed more reliably.

For systematic improvement I maintain an eval set of input/expected-output pairs, run new prompt versions against it, and track metrics. I also log production inputs and periodically sample failures into the eval set.

Q10. Compare embedding models and how you choose one.

Answer: Considerations:

- **Quality on your domain** — MTEB benchmarks are a starting point, not the final answer. Always evaluate on your retrieval task with your data.
- **Dimensionality** — higher dims (1536, 3072) carry more signal but cost more storage and slower search. Matryoshka embeddings let you truncate without retraining.
- **Context length** — short-context models (512 tokens) force aggressive chunking. Long-context embedders (8k+) preserve more semantics per chunk.
- **Multilingual support** — if users query in multiple languages.

- **Cost and latency** — API embeddings (OpenAI `text-embedding-3-large`, Cohere) vs. self-hosted (BGE, E5, Nomic). For high-volume offline indexing, self-hosted often wins on cost.
- **Updateability** — if you re-embed periodically, having control over the model matters.

I always benchmark 2-3 candidates on a labeled retrieval set using recall@k and NDCG before committing.

Q11. Vector search internals — HNSW, IVF, and trade-offs.

Answer: Brute-force k-NN is $O(N \cdot d)$ per query — fine up to ~100k vectors, painful at millions.

- **HNSW (Hierarchical Navigable Small World)** — graph-based, layered. Excellent recall and latency, but high memory (the graph structure is large) and slow build time. Most production systems default to HNSW.
- **IVF (Inverted File Index)** — partitions vectors into clusters; query probes nearest clusters. Lower memory but lower recall unless you probe many clusters (which hurts latency).
- **IVF-PQ (Product Quantization)** — IVF plus compressed vectors. Massive memory savings at moderate recall cost. Used at very large scale (Spotify, FAISS examples).
- **ScaNN** — Google's anisotropic quantization, very strong on benchmarks.

Knobs that matter: `M` and `efConstruction` for HNSW; `nlist`/`nprobe` for IVF. I evaluate on a held-out query set measuring recall@k vs. latency and pick the operating point.

For metadata filtering at scale, watch for pre-filter vs. post-filter behavior — pre-filter is usually faster when filters are selective.

Q12. How do you evaluate an LLM application beyond eyeballing outputs?

Answer: A layered approach:

1. **Unit-level (deterministic)** — schema validation, presence of required fields, citation format.
2. **Reference-based** — for tasks with ground truth: exact match, BLEU/ROUGE (limited), semantic similarity via embeddings.
3. **LLM-as-judge** — for open-ended tasks, an LLM (often a stronger one) scores outputs on rubrics: faithfulness, relevance, coherence, completeness. I calibrate the judge against human ratings on a small set.
4. **RAG-specific (RAGAS)** — faithfulness (no hallucination), answer relevance, context precision/recall.
5. **A/B in production** — task success rates, thumbs up/down, downstream business metrics.

6. **Adversarial/red-team set** — known failure modes (PII, prompt injection, off-topic).

I track these continuously and gate releases on regressions, not just averages — I look at per-segment performance to catch silent failures.

2. MLOps & Cloud (Azure ML, MLflow, CI/CD for ML, Model Monitoring)

Q1. Walk through an Azure ML setup for a team of 5 data scientists.

Answer: I'd structure it as:

- **Workspace** per environment (dev, staging, prod) with separate resource groups, isolated by RBAC.
- **Compute** — shared CPU/GPU clusters with autoscale; per-user compute instances for notebooks; serverless compute for ad-hoc jobs.
- **Datastores** pointed at ADLS Gen2 with managed identity (no key sprawl).
- **Data assets and environments** versioned in the workspace; environments built from conda specs or Dockerfiles, registered as immutable.
- **Component-based pipelines** (YAML or SDK v2) — each step is a reusable component with typed inputs/outputs.
- **Model registry** — models registered with metadata (training run ID, dataset version, metrics) and stages (Development → Staging → Production).
- **Endpoints** — managed online endpoints for real-time, batch endpoints for scoring jobs, with blue-green deployments.
- **CI/CD** — Azure DevOps or GitHub Actions running tests, training, evaluation gate, registration, and deployment.

The principle: notebooks for exploration, components and pipelines for anything that needs to be reproducible.

Q2. How does MLflow fit alongside Azure ML, and what does it actually give you?

Answer: Azure ML uses MLflow as its native tracking backend, so when you log with `mlflow.log_metric`, it shows up in the AML UI automatically. MLflow gives you four things:

1. **Tracking** — runs, parameters, metrics, artifacts, code version.
2. **Projects** — reproducible packaging (less relevant in AML where components fill this role).
3. **Models** — a standard format with flavors (sklearn, pytorch, pyfunc) so the same model can be deployed anywhere.
4. **Registry** — versioned models with stage transitions.

In practice I use MLflow tracking and model logging extensively, register models in the AML registry (which is MLflow-compatible), and deploy via AML endpoints — getting MLflow portability without lock-in.

Q3. Design a CI/CD pipeline for an ML model with a retraining cadence.

Answer: Two pipelines: code CI and model CI/CD.

Code CI (on PR):

- Lint, type check, unit tests for data processing and feature code.
- Integration test on a tiny dataset.
- Build and push training container.

Model CI/CD (on merge to main, or scheduled, or triggered by drift):

1. **Data validation** — schema, range, freshness checks (Great Expectations or similar).
2. **Train** — on the registered pipeline; log everything to MLflow.
3. **Evaluate** — against a versioned holdout set; compare against current production model (champion-challenger).
4. **Quality gate** — automated checks: must beat champion by X%, must pass fairness/segment checks, no regression on critical slices.
5. **Register** — if passed, register with a "candidate" tag.
6. **Deploy to staging** — managed online endpoint, run shadow traffic.
7. **Promote to prod** — blue-green or canary; auto-rollback on health failure.
8. **Post-deploy validation** — smoke test, monitor key metrics for 1-24 hours.

Triggers: code change, scheduled retrain, drift alert, or business-event-driven.

Q4. What do you monitor on a model in production?

Answer: Four layers:

1. **Operational** — latency (p50, p95, p99), throughput, error rate, resource utilization. Standard SRE concerns.
2. **Data quality** — input schema conformance, null rates, range violations, unexpected categorical values. Failures here are loud — alert immediately.
3. **Drift** — covariate drift (input distribution change) via PSI, KS test, or Wasserstein distance per feature; concept drift (relationship between X and y) which requires ground truth.

4. **Performance** — actual metric (accuracy, RMSE) when labels arrive; proxies (prediction distribution stability, confidence drop) when they don't.

Tooling: Azure ML's built-in data drift monitors, or custom dashboards in Application Insights / Grafana. I tier alerts — page on operational failure, ticket on drift, dashboard for slow trends.

The non-obvious bit is closing the loop: when drift fires, what's the action? Auto-retrain, manual investigation, or just notify? I define that per use case.

Q5. How do you handle data and model versioning together?

Answer: They have to be linked or reproducibility breaks. My approach:

- **Data versioning** — datasets registered in Azure ML with version numbers, backed by immutable paths in ADLS (timestamped or hashed folder names). Delta Lake handles this elegantly via time travel.
- **Code versioning** — git SHA logged with every training run.
- **Model versioning** — each registered model carries metadata: training run ID → MLflow run → params, metrics, code SHA, dataset version.

When someone asks "why did this model predict X?", I can trace from the prediction back to the model version, the training run, the exact data snapshot, and the code commit. That lineage is the deliverable.

Q6. How do you do shadow deployment vs canary vs blue-green?

Answer:

- **Shadow** — new model receives copies of production traffic but its responses aren't returned to users. Best for validating performance and behavior on real inputs with zero risk. Cost: double inference.
- **Canary** — route a small percentage (1-10%) of real traffic to the new model. Real impact measurement but real risk for that slice. Good for measuring user-facing metrics.
- **Blue-green** — two identical environments; flip traffic from blue (old) to green (new) all at once after validation. Fast rollback by flipping back. Doubles infrastructure cost temporarily.

I usually shadow first (catch silent regressions), then canary (catch user-facing issues), then full rollout via blue-green for the cutover.

Q7. Explain feature store and when it's worth the investment.

Answer: A feature store solves three problems: (1) training-serving skew, (2) feature reuse across teams, (3) point-in-time correctness for time-series features.

It separates feature *definition* from feature *consumption*. Features are computed once, stored in an offline store (for training) and online store (for low-latency serving), and accessed by a unified API.

Worth it when:

- Multiple teams build models on overlapping features.
- You have time-series or aggregation features (rolling windows) where point-in-time correctness is hard.
- Serving latency requires precomputed features.

Not worth it when:

- One team, a few features, batch scoring only — a Delta table and good pipelines are enough.

In Azure I've used Feathr and the managed feature store in Azure ML. Databricks Feature Store is also a common choice.

Q8. How do you make ML pipelines reproducible?

Answer: Reproducibility has four pillars:

1. **Data** — versioned, immutable snapshots referenced by ID.
2. **Code** — git SHA pinned; no `main` references in production.
3. **Environment** — pinned dependencies, containerized; same base image across train and serve.
4. **Randomness** — seeded RNGs (numpy, framework, OS-level); deterministic ops where GPU non-determinism matters.

Even with all four, exact reproducibility on GPU is sometimes impossible — I aim for *statistical* reproducibility (same metrics within tolerance) and document any sources of non-determinism.

3. Azure AI Foundry

Q1. What is Azure AI Foundry and how is it different from Azure ML?

Answer: Azure AI Foundry (formerly Azure AI Studio) is Microsoft's unified platform for

building generative AI applications — agents, chatbots, RAG systems, multi-modal apps. It sits on top of Azure ML and Azure OpenAI but is oriented around GenAI workflows.

Key differences:

- **Azure ML** is general-purpose ML — training, registration, deployment of any model type.
- **AI Foundry** is GenAI-first — model catalog (Azure OpenAI, Llama, Mistral, Phi, etc.), prompt flow, agent builder, evaluation harness for LLMs, content safety integration, and grounding-data connectors.

In practice Foundry is where I build LLM apps; AML is where I do tabular ML or train custom models. They share underlying compute and registry concepts.

Q2. Walk through Prompt Flow in AI Foundry.

Answer: Prompt Flow is a visual + code orchestration tool for LLM pipelines. A flow is a DAG of nodes — Python functions, LLM calls, prompt templates, embeddings, vector lookups.

Typical RAG flow: input → embed query → vector search → reranker → prompt template (with retrieved context) → LLM call → output. Each node is independently testable. Variants let you A/B prompts or models in the same flow.

The value: you can version flows in git, run bulk evaluations against a dataset, view per-node traces, and deploy the entire flow as a managed endpoint. It's roughly LangChain + LangSmith but native to Azure.

Q3. How do you handle evaluation in AI Foundry?

Answer: Foundry's evaluation framework supports:

- **Built-in metrics** — groundedness, relevance, coherence, fluency, similarity (LLM-as-judge using a configured judge model).
- **Risk and safety metrics** — hateful content, sexual content, violence, self-harm, protected material, indirect attacks (prompt injection).
- **Custom metrics** — Python functions or custom prompt evaluators you define.

I attach an evaluation dataset, run the target flow over it, and Foundry produces a report with per-row scores and aggregate metrics. I gate deployments on these and track regressions across versions.

Q4. How does grounding work in Foundry, and how do you reduce hallucination?

Answer: Foundry supports grounding via "Add your data" — connecting to Azure AI Search, Cosmos DB, blob storage, or other sources. The retrieved content is injected into the prompt, and the response includes citations.

To reduce hallucination:

- **Strict system prompt** — instruct the model to answer only from provided context, return "I don't know" otherwise.
 - **High-quality retrieval** — better embeddings, hybrid search, reranking. Bad context → bad answers.
 - **Groundedness evaluation** in CI — fail builds if groundedness score drops.
 - **Citation requirement** — ask the model to cite chunk IDs; downstream, validate that cited chunks exist.
 - **Confidence thresholding** — if no high-similarity chunks are retrieved, route to a fallback path rather than letting the model improvise.
-

Q5. How would you build a multi-agent application in AI Foundry?

Answer: Foundry's agent service supports tool use, code interpreter, file search, and connected actions. For multi-agent, I'd design:

- **Orchestrator agent** — receives the user request, decomposes, and delegates.
- **Specialist agents** — domain experts (e.g., one for SQL, one for document search, one for action execution).
- **Shared state** — conversation history and intermediate results, often in a thread or external store.
- **Tool definitions** — each agent has a constrained set of tools; the orchestrator's "tools" are calls to other agents.

Key design choices: how chatty are the agents (token cost), how do you prevent infinite loops (max turns, explicit termination conditions), and how do you trace and debug (every call logged with inputs, outputs, latency). For complex graphs I'd reach for LangGraph or Semantic Kernel patterns on top.

Q6. How do you handle content safety and responsible AI in Foundry?

Answer: Foundry integrates Azure AI Content Safety for input and output filtering — categories include hate, violence, sexual, self-harm, jailbreak detection, and protected-material detection. I configure thresholds per category and decide whether to block, warn, or annotate.

Beyond content safety:

- **PII detection and redaction** before logging and before sending to the model where possible.
- **Prompt shield** for prompt injection / indirect injection attacks (especially when retrieving from untrusted sources).
- **Output validation** — schema, allowlists, regex for sensitive patterns.
- **Audit logging** — every prompt and response stored for review with retention policies.
- **Human review** for high-stakes outputs.

I treat safety as a layered system — no single mechanism is sufficient.

4. Azure Data & AI Solutions

Q1. How do you choose between Synapse, Databricks, and Fabric for a new project?

Answer: Rough decision logic:

- **Microsoft Fabric** — when the org is Power BI-heavy, wants unified SaaS, and the workload is medium-scale with mixed BI + lakehouse needs. Easy onboarding, OneLake is genuinely useful, but it's newer and has rougher edges for very large workloads.
- **Azure Databricks** — when you need top-tier Spark performance, advanced ML/AI workloads, Delta Lake at scale, and your team has Spark experience. Best-in-class for data engineering and ML.
- **Synapse Analytics** — historically the answer for dedicated SQL pools (now Synapse Dedicated SQL); for new projects, Microsoft is steering toward Fabric. I still see Synapse where there's existing investment.

I also weigh: existing skills, governance needs (Purview integrates with all three), cost model (Fabric capacity vs Databricks DBU), and the BI consumption pattern.

Q2. Sketch a reference architecture for a real-time analytics + AI solution on Azure.

Answer: Event ingestion → stream processing → lakehouse → serving + AI:

- **Ingestion** — Event Hubs or IoT Hub for streaming events; ADF or copy jobs for batch sources.
- **Stream processing** — Azure Stream Analytics or Spark Structured Streaming (Databricks/Fabric) for windowed aggregations, joins, enrichment.
- **Storage** — ADLS Gen2 in medallion layout (bronze/silver/gold) with Delta or Iceberg tables.

- **Batch processing** — Databricks or Fabric notebooks for heavier transformations.
- **Serving** — gold-layer Delta tables exposed via SQL endpoint to Power BI (DirectLake in Fabric); hot path via Cosmos DB or Azure SQL for sub-second reads.
- **AI layer** — Azure ML / AI Foundry models consumed via online endpoints; Azure AI Search for vector + keyword retrieval.
- **Governance** — Purview for lineage, classification, glossary.
- **Orchestration** — ADF or Fabric pipelines.

Security: managed identities throughout, private endpoints, role-based access on the workspace and data plane.

Q3. How would you design a RAG solution on Azure end-to-end?

Answer: Components:

- **Source documents** in SharePoint, ADLS, or other systems.
- **Ingestion pipeline** (ADF or Fabric) — pulls documents, extracts text (Form Recognizer / Document Intelligence for PDFs and images), chunks, embeds via Azure OpenAI embeddings.
- **Vector store** — Azure AI Search (it supports hybrid: BM25 + vector + semantic ranker, which is a strong default), or Cosmos DB for NoSQL with vector index, or a dedicated DB like Pinecone.
- **App layer** — Azure Functions or App Service hosting the orchestrator; or a Foundry-deployed Prompt Flow.
- **LLM** — Azure OpenAI (gpt-4o, gpt-4-turbo) with content safety enabled.
- **Telemetry** — Application Insights for traces; evaluation runs in Foundry.

Security: managed identities, private endpoints on Search and OpenAI, RBAC for document-level access control (I propagate ACLs into Search via security filters so users only retrieve documents they're authorized to see).

Q4. What governance considerations matter for an Azure data and AI platform?

Answer:

- **Catalog and lineage** — Purview scans ADLS, SQL, Power BI, Fabric; tracks lineage across pipelines.
- **Classification** — automatic discovery of PII, financial data, etc.; sensitivity labels propagate.
- **Access control** — RBAC + ABAC + ACLs at storage level; row-level security in serving layer; OLS in Power BI.

- **Data quality** — automated checks at pipeline boundaries; Great Expectations or built-in Fabric data quality.
- **AI governance** — model registry with approval workflows; evaluation reports stored; content safety logs; responsible AI documentation.
- **Cost governance** — tags, budgets, alerts; reservation/capacity sizing reviewed monthly.
- **Compliance** — encryption at rest (CMK where required), encryption in transit, audit logs to Log Analytics, retention policies.

The hardest part is consistent enforcement — I push for policy-as-code (Azure Policy) and automated checks rather than reliance on documentation.

Q5. How do you optimize cost on a large Azure data platform?

Answer: Common wins:

- **Right-size compute** — autoscale, scheduled scale-down for dev/test, spot/low-priority where tolerated.
- **Reserved instances / savings plans** for predictable workloads.
- **Storage tiering** — hot/cool/archive for ADLS; lifecycle policies to age out data.
- **File optimization** — compact small files (the classic problem); optimize Delta tables; partition correctly.
- **Query optimization** — avoid full scans; pushdown filters; materialized views for repeated heavy queries.
- **Eliminate dual storage** — Fabric's OneLake shortcuts and Databricks Unity Catalog reduce data duplication.
- **Monitor egress** — cross-region traffic is expensive.
- **AI cost** — cache embeddings; use cheaper models for routing/classification, reserve top-tier models for hard tasks; provisioned throughput when usage is predictable.

I run a monthly cost review and tag-based attribution back to teams so costs are visible to the people creating them.

Q6. How do you migrate an on-prem data warehouse to Azure?

Answer: Phased approach:

1. **Assess** — inventory of sources, schemas, jobs, consumers, SLAs. Identify what to migrate vs deprecate.

2. **Target architecture** — pick the destination (Fabric, Synapse, Databricks SQL); design medallion layout, security model, naming conventions.
3. **Pilot** — one domain end-to-end: ingest → transform → consume in Power BI. Prove the pattern, catch issues.
4. **Lift-and-shift** for low-risk objects; **re-engineer** for objects with technical debt. SSIS packages can run on ADF SSIS IR for short-term parity.
5. **Parallel run** — run old and new in parallel, compare outputs row-by-row on key datasets until reconciliation is clean.
6. **Cutover** with consumer communication; keep rollback path for 30+ days.
7. **Decommission** old system after dual-run confidence.

Risks I plan for: data type mismatches, timezone handling, NULL semantics differences, BI report breakage, and skills gap.

5. DAX — Measures and Basics

Q1. Explain the difference between a calculated column and a measure.

Answer: A **calculated column** is computed row-by-row at refresh time and stored in the model. It has a row context — each row sees its own values. Use it when you need a value that's fixed per row (a category derived from existing fields) and you'll filter or group by it.

A **measure** is computed at query time within whatever filter context the visual provides. It doesn't store values — it stores the formula. Use it for aggregations and anything that should respond to slicers and filters.

Rules of thumb: if it should change as the user slices, it's a measure. If it's a property of a row that you might filter on, it's a calculated column. When in doubt, prefer a measure — calculated columns increase model size and refresh time.

Q2. What is filter context vs row context? How does CALCULATE bridge them?

Answer: **Row context** exists when you're iterating over a table — calculated columns and iterator functions (SUMX, FILTER) provide it. It means "the current row." Row context does NOT automatically filter other tables; you need a relationship plus context transition.

Filter context is the set of filters applied to the model — coming from slicers, visual axes, page filters, or explicit DAX. Measures execute in filter context.

CALCULATE does two things: (1) it can modify the existing filter context (add, remove, override filters), and (2) it performs **context transition** — converting the current row context into an

equivalent filter context. This is why `CALCULATE([Measure])` inside a calculated column "sees" only the current row's data.

Misunderstanding context transition is the source of most DAX bugs I encounter.

Q3. Walk me through writing a year-over-year sales measure.

Answer: Assuming a proper Date table marked as a date table with a continuous date range:

DAX

```
Sales := SUM(FactSales[Amount])

Sales LY :=
CALCULATE(
    [Sales],
    SAMEPERIODLASTYEAR( DimDate[Date] )
)

YoY Growth % :=
DIVIDE( [Sales] - [Sales LY], [Sales LY] )
```

Things to check:

- A real Date table (not the auto date/time), continuous, marked as date table.
 - Relationship between FactSales[Date] and DimDate[Date].
 - Use `DIVIDE` to handle division by zero gracefully.
 - For YTD: `TOTALYTD([Sales], DimDate[Date])`.
 - For non-standard fiscal years, pass year-end-date to TOTALYTD or use a custom calendar with offset columns.
-

Q4. What does RANKX do, and what's a common pitfall?

Answer: `RANKX` ranks each item by evaluating an expression over a table. Typical use:

DAX

```
Product Rank by Sales :=
RANKX( ALL( DimProduct[Product] ), [Sales] )
```

The `ALL` removes the existing product filter so RANKX sees every product and can produce a meaningful ranking. Without it, every row is ranked against itself = 1.

Pitfalls:

- **Forgetting ALL** — most common bug.
 - **Ranking at the wrong grain** — if you want a rank within a category, use `ALLEXCEPT(DimProduct, DimProduct[Category])`.
 - **Ties** — third argument controls dense/skip ranking; fourth controls ascending/descending.
 - **Totals row** — by default RANKX evaluates at the total level too, often producing a weird "1" or blank. Wrap with `IF(HASONEVALUE(...), RANKX(...))` to suppress.
-

Q5. Difference between SUMX and SUM, ALL, ALLEXCEPT, and REMOVEFILTERS.

Answer:

- **SUM(column)** — aggregates a column in current filter context. Simple.
- **SUMX(table, expression)** — iterates the table row by row, evaluates expression in row context, sums results. Use when the expression needs per-row calculation: `SUMX(Sales, Sales[Qty] * Sales[Price])`.
- **ALL(table or column)** — removes all filters from the argument.
- **ALLEXCEPT(table, col1, col2...)** — removes all filters from the table except those listed.
- **REMOVEFILTERS** — functional equivalent to ALL but more readable when used as a CALCULATE modifier; signals "I'm removing filters" rather than "I'm returning a table."

Modern DAX style: use `REMOVEFILTERS` inside `CALCULATE` for clarity; use `ALL` when you need a table expression.

Q6. Explain variables (VAR) and why they matter.

Answer: `VAR` defines a named value evaluated once and reused. Three reasons they matter:

1. **Performance** — `VAR` is computed once, not per reference. Replacing repeated subexpressions with a variable can dramatically speed up complex measures.
2. **Readability** — complex measures broken into steps.
3. **Context capture** — a `VAR` captures its value in the context where defined; subsequent CALCULATES don't change it. This is incredibly useful when comparing "current row's value" to a recomputed value.

DAX

```
Margin % :=  
VAR Revenue = [Sales]  
VAR Cost = [Total Cost]  
RETURN DIVIDE( Revenue - Cost, Revenue )
```

I treat variables as a default — almost every non-trivial measure uses them.

Q7. What is the difference between USERELATIONSHIP and TREATAS?

Answer: Both manipulate relationships, but differently.

USERELATIONSHIP activates an inactive physical relationship inside a **CALCULATE**. Used when a fact table has multiple date columns (order date, ship date) and you want a measure based on ship date while order date is the active relationship:

DAX

```
Sales by Ship Date :=  
CALCULATE( [Sales], USERELATIONSHIP( FactSales[ShipDate], DimDate[Date] ) )
```

TREATAS applies a table expression as a filter on another table's columns — effectively creating a virtual relationship at query time. Used when no physical relationship exists or when you want a relationship only for a specific calculation:

DAX

```
Budget vs Actual :=  
CALCULATE(  
    [Actual],  
    TREATAS( VALUES( BudgetTable[Category] ), Product[Category] )  
)
```

TREATAS is powerful but slower than physical relationships at scale — I prefer to model relationships physically where reasonable.

Q8. How do you optimize a slow DAX measure?

Answer: Step by step:

1. **Capture the query** in DAX Studio (Performance Analyzer in Power BI gives the DAX).

2. **Look at Storage Engine (SE) vs Formula Engine (FE) split** — high FE time means CPU-bound DAX logic; high SE means scans. Different remedies for each.
3. **Reduce iterator work** — replace SUMX over large tables with SUM where possible; filter early inside the iterator.
4. **Avoid bidirectional relationships** in large models — they cause expensive scans.
5. **Watch for context transitions** in loops — they materialize tables.
6. **Use variables** to avoid recomputation.
7. **Pre-aggregate** — if a measure repeatedly sums millions of rows, create an aggregation table (or use Power BI aggregations) for the common grain.
8. **Reduce cardinality** — split high-cardinality columns; remove unused columns (model size affects scan cost).
9. **Star schema discipline** — measures over star schemas are dramatically faster than over snowflakes or wide flat tables.

DAX Studio's Server Timings and Query Plan are essential — never optimize blind.

6. Azure Data Factory (ADF)

Q1. Explain Integration Runtimes and when to use which.

Answer: Three types:

- **Azure IR** — managed, runs in Azure; for cloud-to-cloud activities. Auto-scales, region-aware. Default choice.
- **Self-hosted IR** — installed on a VM (on-prem or in your VNet); used to access on-prem data or data behind a firewall. Can be scaled out across multiple nodes.
- **Azure-SSIS IR** — dedicated cluster to lift-and-shift SSIS packages; you provision node size and count.

I pick Azure IR by default; self-hosted only when there's no other way to reach the source; SSIS IR only for migration scenarios where rewriting isn't feasible short-term.

Q2. Difference between Mapping Data Flows, Copy Activity, and Wrangling Data Flows.

Answer:

- **Copy Activity** — fast, simple data movement between source and sink with optional schema mapping. No transformations beyond column mapping. Use it for ingestion.
- **Mapping Data Flow** — visual, code-free transformation that runs on Spark behind the scenes. Joins, aggregations, derived columns, conditional splits. Use it for transformations

when you don't want to write Spark code yourself.

- **Wrangling Data Flow** — Power Query in ADF, for citizen-developer-style data prep. Less common in production.

For heavy transformation logic I prefer Databricks notebooks called from ADF — more control, easier to test, better performance tuning.

Q3. How do you parameterize pipelines for reuse?

Answer: Three layers of parameters:

1. **Pipeline parameters** — passed at trigger time (e.g., source system, run date).
2. **Dataset parameters** — make datasets generic (file path, table name driven by parameters).
3. **Linked service parameters** — connection strings or container names parameterized.

A metadata-driven framework typically:

- Stores configuration (sources, sinks, schedules, columns) in a SQL table.
- A parent pipeline reads metadata, iterates with `ForEach`, calls a child pipeline per entity passing parameters.
- The child pipeline is generic — same logic, different config.

This lets you onboard a new source by inserting a row, not building a new pipeline.

Q4. How do you handle incremental loads and CDC in ADF?

Answer: Several patterns depending on the source:

- **Watermark column** — store last loaded value (`max(updated_at)`) in a control table; next run pulls rows where `updated_at > watermark`. Works for sources with reliable change timestamps.
- **Native CDC** — SQL Server CDC, Azure SQL change tracking; ADF has dedicated CDC connectors.
- **Snapshot diff** — compare current snapshot to previous; expensive but works when no change marker exists.
- **Log-based replication** — Debezium, Fivetran, or Azure DMS for high-volume sources.

I always store load metadata (watermark, row counts, run ID, status) in a control table so reruns and reconciliation are possible. And I make pipelines **idempotent** — rerunning the same window should produce the same result.

Q5. How do you handle error handling and retries?

Answer: Activity-level: `retry`, `retryIntervalInSeconds`, and `timeout`. For transient failures (network blips) this is usually enough.

Pipeline-level: each activity has `success`, `failure`, `completion`, `skipped` paths. I wire failure paths to logging activities that write to a control table and send alerts.

Patterns I use:

- **Try-catch via execute pipeline** — call a child pipeline; on failure, an error-handling activity logs the error and decides whether to fail or continue.
- **Dead-letter** — bad rows written to a quarantine container with original payload and error details.
- **Idempotent design** — every pipeline can be safely rerun. No partial writes; use staging + atomic move/swap.
- **Monitoring** — Log Analytics integration; KQL alerts on failure counts; SLA dashboards.

I never rely on the "Send email on failure" pattern as the only safety net.

Q6. How do you implement CI/CD for ADF?

Answer: Two common patterns:

1. **Git integration with `adf_publish` branch** — dev factory is git-connected; publishing generates ARM templates in `adf_publish`; release pipeline deploys ARM to staging and prod factories with parameter overrides.
2. **ADF utilities / npm package** — generate ARM templates from `main` branch directly, removing the need for `adf_publish`. Better for pure CI flows.

I parameterize linked services and global parameters per environment. The deploy pipeline:

- Stops triggers in target factory.
- Deploys ARM.
- Updates global parameters via PowerShell.
- Restarts triggers.
- Runs smoke tests.

For testing, I use pipeline tests via the REST API and validate against a test dataset.

Q7. How do you secure ADF — credentials, networking, data?

Answer:

- **Credentials** — never inline. Use Azure Key Vault references in linked services; ADF reads from Key Vault via managed identity.
 - **Managed identities** for connecting to Azure resources (SQL DB, Storage) so there are no passwords at all.
 - **Networking** — managed VNet for ADF gives private endpoints to Azure resources; data plane traffic doesn't traverse the public internet. Self-hosted IR for on-prem.
 - **RBAC** — least-privilege roles on the factory (contributor only for those who need it; reader for monitoring).
 - **Logging** — all activity runs to Log Analytics; access logs to a SIEM.
 - **Data** — encryption at rest (storage), TLS in transit (default), customer-managed keys where regulatory requirements demand.
-

Q8. ADF vs Synapse Pipelines vs Fabric Pipelines — what's the difference and when do you use which?

Answer: Architecturally they're close cousins — Synapse Pipelines and Fabric Data Pipelines are essentially the ADF engine embedded in those platforms with platform-specific activities (Synapse notebook, Fabric notebook, Spark job definition).

- **ADF** — standalone factory; richest connector library; mature; uses ARM/git integration; good for hybrid scenarios.
- **Synapse Pipelines** — integrated with Synapse workspace; one-pane access to Spark and SQL pools.
- **Fabric Pipelines** — integrated with Fabric capacity; uses Fabric items (lakehouses, warehouses, notebooks); no separate provisioning.

For new builds today my default is: Fabric Pipelines if the org is committed to Fabric; ADF if hybrid or multi-cloud or the connector list matters; Synapse Pipelines mostly for existing Synapse investments.

7. Azure Data Lake Storage (ADLS Gen2)

Q1. What makes ADLS Gen2 different from Blob Storage?

Answer: ADLS Gen2 is Blob Storage with a **hierarchical namespace (HNS)** enabled.

Differences:

- **True directories** — rename and delete on a directory is a single atomic metadata operation, not a loop over blobs. Massive performance win for big-data workloads.
- **POSIX-like ACLs** — fine-grained permissions per file and directory, on top of RBAC.
- **Optimized for analytics** — Hadoop-compatible (`abfs://` driver), Spark-friendly.

You also pay for HNS — slightly higher transaction costs. For pure object storage (images, backups), regular blob is fine. For data lakes, always Gen2.

Q2. How do you organize a data lake — folder structure, partitioning, file formats?

Answer: Medallion architecture is my default:

- **Bronze** — raw ingested data, append-only, often the original format (JSON, CSV). Partitioned by ingestion date.
- **Silver** — cleansed, conformed, typed. Delta or Parquet. Partitioned by business-meaningful columns.
- **Gold** — business-ready aggregates and dimensional models for BI.

Within each layer: `/<layer>/<domain>/<entity>/<partition_columns>/`.

File formats: Parquet for analytics (columnar, compressed); Delta or Iceberg for ACID, time travel, schema evolution. Avoid CSV/JSON beyond raw bronze.

Partitioning: by columns you actually filter on, with cardinality keeping partition counts in the thousands (not millions). Date is the most common; avoid high-cardinality partition keys.

File size: target 128 MB–1 GB compacted Parquet files. Small files are the #1 performance killer — schedule compaction (`OPTIMIZE` in Delta).

Q3. How do you secure ADLS Gen2?

Answer: Layered security:

1. **Network** — private endpoints, firewall rules, service endpoints; disable public access where possible.
2. **Identity** — Entra ID auth via managed identities or service principals; no account keys in code.
3. **RBAC** — at the account or container level (Storage Blob Data Reader/Contributor).
4. **POSIX ACLs** — fine-grained file/directory permissions for principals not covered by RBAC, or for granular sharing.

5. **Encryption** — at rest by default with Microsoft-managed keys; customer-managed keys (CMK) for regulated workloads.
6. **Soft delete and versioning** — protect against accidental deletes; immutable storage for compliance.
7. **Audit** — diagnostic logs to Log Analytics; alerts on anomalous access patterns.

Common pitfall: layering ACLs on top of RBAC and forgetting the effective permission is the union — accidentally over-permissive.

Q4. How does RBAC interact with POSIX ACLs?

Answer: They're evaluated together. RBAC at the storage account level (e.g., Storage Blob Data Contributor) grants access across the account. POSIX ACLs grant access at directory/file level.

A user gets access if **either** mechanism grants it. So if you've given Storage Blob Data Reader at the account, fine-grained ACLs denying access won't help — the RBAC role wins. Best practice: use a minimal RBAC role (e.g., "Reader" for management plane) and rely on ACLs for data-plane permissions when you need fine control.

For most teams I just use RBAC at the container level and avoid ACLs — they're hard to manage at scale. ACLs become necessary when sharing specific paths across organizational boundaries.

Q5. How do you handle large-scale ingestion into ADLS efficiently?

Answer:

- **Parallel writes** — partition source data and write in parallel from multiple workers.
- **Right file size** — avoid creating millions of tiny files; buffer and write larger files.
- **Choose the right API** — `abfs://` driver from Spark/Hadoop; native SDK for direct apps. Avoid REST loops for high volume.
- **Throttling** — ADLS has per-account ingress and IOPS limits; if you hit them, spread across containers or accounts, or request limit increases.
- **Network** — private endpoint or service endpoint to keep traffic on the Azure backbone.
- **Compression** — gzip or snappy for Parquet (snappy is the usual default — faster decode).

For one-time bulk migrations: AzCopy, Azure Data Box for petabyte-scale.

Q6. What is the lifecycle management policy in ADLS, and how do you use it?

Answer: Lifecycle management lets you automate tier transitions and deletions based on rules.

Tiers:

- **Hot** — frequent access; highest storage cost, lowest access cost.
- **Cool** — infrequent (>30 days); lower storage cost, higher access cost.
- **Cold** — rarely accessed (>90 days); cheaper still.
- **Archive** — rarely-needed retention; cheapest storage but hours to rehydrate.

Typical policy: bronze raw files → cool after 30 days → archive after 180. Silver/gold stay hot. Logs → archive after 90 days. Delete temp data after 7 days.

Rules are JSON, scoped by prefix or blob tags. I always test on a small prefix before applying broadly — transitions are billable operations and archive rehydration has cost and latency.

8. Microsoft Fabric

Q1. What is Microsoft Fabric and how does it relate to existing Azure services?

Answer: Fabric is a SaaS analytics platform that unifies data engineering, data science, real-time intelligence, data warehousing, and BI on top of a single storage layer called **OneLake**. It bundles capabilities that previously lived in Synapse, Power BI, Data Factory, and parts of Azure ML.

Key concept: **one capacity, multiple workloads**. You buy F-SKU capacity and consume it across Spark notebooks, Warehouse SQL queries, Power BI reports, real-time eventstreams, etc. Storage is unified in OneLake — all data is stored as Delta Parquet, no duplication.

It's *not* a replacement for everything. Heavy data engineering still benefits from Databricks. Custom ML at scale is still Azure ML / Databricks. But for an integrated analytics + BI experience with Power BI at the center, Fabric is increasingly the path of least resistance.

Q2. Explain OneLake and shortcuts.

Answer: **OneLake** is the single logical data lake for an entire Fabric tenant — "OneDrive for data." Every workspace gets a folder; every lakehouse, warehouse, and KQL database stores data inside OneLake automatically, in Delta Parquet.

Shortcuts are pointers — symbolic links — to data stored elsewhere (other OneLake locations, ADLS Gen2, S3, Dataverse). They appear in your lakehouse as if the data lived there, but there's no copy. This enables:

- **One copy of data** — multiple workspaces consume the same physical data without duplication.
- **Cross-cloud** — bring S3 data into Fabric without moving it.

- **Multi-tenant patterns** — central data domain owners publish; consumers shortcut.

Caveats: shortcut access still requires permissions on the source; latency depends on source location; write-through behavior varies by source type.

Q3. What is DirectLake mode in Power BI, and how is it different from Import and DirectQuery?

Answer:

- **Import** — data copied into Power BI's in-memory engine (VertiPaq). Fast queries, but requires refresh; size limited by capacity.
- **DirectQuery** — every query goes to the source system at runtime. No refresh needed, always fresh, but query performance depends on source.
- **DirectLake** — Fabric-only. The Power BI engine reads Delta Parquet files directly from OneLake, page by page, into memory on demand. No import refresh, no per-query round trip to a source.

DirectLake gives import-like performance with live data, provided the underlying lakehouse/warehouse data is well-modeled (good star schema, proper data types, partitioned). It falls back to DirectQuery for some scenarios (cross-source queries, complex M, RLS combined with calculation groups in older versions), so you should test before assuming you're always in DirectLake mode.

Q4. Lakehouse vs Warehouse in Fabric — when do you use which?

Answer: Both store data as Delta in OneLake. The differences:

- **Lakehouse** — Spark-first; multiple file formats supported in the Files area; notebooks are the primary tool; schema-on-read flexibility. Best for data engineering, ML, semi-structured data, large transformations.
- **Warehouse** — SQL-first; T-SQL DDL/DML; transactional multi-table writes; familiar to BI developers. Best for traditional dimensional modeling and SQL-heavy consumers.

A mature architecture often uses both: lakehouse for bronze/silver (Spark transformations), warehouse for gold (T-SQL stored procs for marts, accessed via Power BI). Cross-querying via the SQL endpoint works seamlessly.

Q5. How do you do capacity planning and cost management in Fabric?

Answer: Fabric uses **Capacity Units (CU)** — a unit of compute consumed by all workloads. SKUs scale from F2 to F2048. Key concepts:

- **Smoothing** — Fabric averages your usage over a 24-hour window; brief spikes are absorbed.
- **Bursting** — workloads can burst above your capacity briefly, but sustained overuse causes **throttling**.
- **Workload attribution** — the Fabric Capacity Metrics app shows which items consume the most CUs.

Practices:

- Right-size for steady-state plus modest headroom; let smoothing absorb spikes.
- Schedule heavy refreshes during low-usage windows.
- Pause non-prod capacities outside business hours.
- Watch out for runaway notebooks — set timeouts.
- Use separate capacities for dev/prod to prevent dev workload from throttling user reports.

I review the capacity metrics app weekly during ramp-up, monthly once stable.

Q6. What is a semantic model in Fabric, and how is it different from a Power BI dataset?

Answer: "Semantic model" is the renamed term for what was called a "dataset" in Power BI — the tabular model (tables, relationships, measures, RLS) that sits between data and reports.

In Fabric, semantic models are first-class items, can live in any workspace, and can be created in three modes: Import, DirectQuery, DirectLake. They can be shared across reports and consumed via XMLA endpoints by tools like DAX Studio, Tabular Editor, or Excel.

Best practice: one semantic model per business domain, multiple thin reports built on it. Avoid one model per report — it creates a maintenance nightmare.

9. ETL / Data Engineering

Q1. ETL vs ELT — which do you choose and why?

Answer:

- **ETL** — transform before loading. Common with traditional data warehouses where storage was expensive and compute lived on a beefy ETL server.

- **ELT** — load raw, then transform in the warehouse/lakehouse using its compute. Standard for modern cloud platforms (Fabric, Databricks, Snowflake).

I default to **ELT** today because:

- Cloud storage is cheap; keeping raw data enables reprocessing without re-ingesting.
- Cloud compute scales horizontally and is elastic.
- SQL-based transformations are easier to maintain than custom ETL code.
- dbt has made ELT pipelines testable, modular, and documented.

ETL is still appropriate when source systems can't tolerate the load of pulling raw data, or for sensitive data that must be masked before landing.

Q2. How do you design idempotent pipelines?

Answer: Idempotency means rerunning a pipeline (with the same parameters) produces the same final state, regardless of how many times you run it. Patterns:

- **MERGE / UPSERT** instead of plain INSERT — keyed by business key.
- **Delete-and-reinsert per partition** — atomic at the partition level. Reload partition for date X means delete data for X, insert new data for X.
- **Idempotent file outputs** — overwrite a path; downstream consumers always read the latest.
- **Transaction-bound writes** — Delta Lake's atomic commits make this clean.
- **Deduplication** — if upstream can re-deliver the same event, dedupe by (event_id, ingest_time) in silver.
- **No side effects in the source** — don't move/delete source files until the load is confirmed (or use control table to track what's processed).

Always design for "what if this runs twice?" — the answer should be "no harm."

Q3. How do you handle slowly changing dimensions (SCDs)?

Answer:

- **Type 0** — never changes (date of birth).
- **Type 1** — overwrite; no history (current address only).
- **Type 2** — keep history; new row per change with effective_from / effective_to / is_current. The most common when you need point-in-time analysis.
- **Type 3** — keep limited history (previous value as a separate column). Used rarely.

- **Type 6** — hybrid of 1, 2, 3.

Implementation in modern data lakes: SCD2 via Delta MERGE — match on business key, when matched and changed, expire current row and insert new row. The `WHEN NOT MATCHED BY SOURCE` clause handles deletes (set `is_current = 0`).

Fact tables join to dimension on the business key for current-state, or via effective dating for historical accuracy.

Q4. How do you handle data quality in pipelines?

Answer: Layered quality framework:

1. **Schema validation** at ingest — column names, types, required fields. Reject malformed files to dead-letter.
2. **Row-level rules** — Great Expectations or custom (not null, ranges, regex, referential integrity). Quarantine bad rows.
3. **Reconciliation** — row count and sum checks between source and target; pipeline fails or alerts if delta exceeds threshold.
4. **Anomaly detection** — daily volume far below expected? Distinct customers dropped 50%? Alert.
5. **Data contracts** — formal agreement between producers and consumers about schema, semantics, freshness, SLAs.

Quality issues are caught earliest at the producer boundary. I push data contracts upstream when politically possible — they prevent silent breakage when source schemas change.

Q5. Describe star schema vs snowflake vs data vault — when to use each?

Answer:

- **Star schema** — one central fact table, dimensions denormalized into single tables. Optimized for analytics; simple for BI users. Default for marts.
- **Snowflake** — dimensions normalized further (e.g., product → category → department as separate tables). Saves storage but slows queries (more joins). Mostly historical; rarely justified now.
- **Data Vault** — hub/link/satellite model. Designed for raw-vault, integration-vault, with full auditability and ease of source addition. Heavy to query directly; usually feeds star schemas downstream.

I default to **star schema for the gold layer**. Bronze is raw, silver is conformed and cleansed (often 3NF-ish), gold is dimensional. Data Vault for very large enterprises with many source systems and strong audit requirements.

Q6. How do you handle late-arriving facts and late-arriving dimensions?

Answer:

- **Late-arriving facts** — fact event arrives but the dimension row for its key doesn't exist yet. Options: (1) reject and reprocess later; (2) insert with a placeholder "unknown" dimension key and update later; (3) inferred member — insert a stub dimension row with just the key, fill in attributes when they arrive.
- **Late-arriving dimensions** — new attributes for an existing dimension arrive after facts already referenced the row. With SCD2, this means inserting historical versions retroactively, which means updating fact-dimension keys.

In practice I avoid retroactive key changes (they're painful) and use a "current key + effective dates" lookup pattern — facts store the business key, and joins resolve the surrogate key by date at query time. This keeps facts immutable.

10. DAX Studio and TOAD

Q1. What do you use DAX Studio for that you can't easily do in Power BI Desktop?

Answer: DAX Studio is essential for serious DAX work:

- **Performance tuning** — Server Timings shows Storage Engine vs Formula Engine time, query plan, scan counts. Power BI's Performance Analyzer gives you the DAX text but DAX Studio shows you *why* it's slow.
- **Running ad-hoc DAX queries** — EVALUATE statements against a model, with full result sets.
- **Connecting to deployed models** — XMLA endpoint connection to a published semantic model in Power BI Service / Fabric for tuning live models.
- **Capturing all queries** sent by Power BI reports while interacting — invaluable when diagnosing a slow report.
- **Exporting query results** to CSV, Excel for verification.
- **Comparing measure variants** side by side.

I install it alongside Power BI Desktop as a non-negotiable.

Q2. Walk me through diagnosing a slow Power BI visual using DAX Studio.

Answer:

1. In Power BI Desktop, open **Performance Analyzer**, refresh, interact with the slow visual, click **Copy query**.
 2. In DAX Studio, connect to the same model, paste the query.
 3. Enable **Server Timings** (the most important toggle).
 4. Run the query.
 5. Look at the totals: total duration, SE time, FE time, SE queries count.
- High SE % with many queries → too many scans, often from bidirectional filters, complex relationships, or unaggregated fact tables. Consider aggregations, simpler model, or reduced cardinality.
 - High FE % → CPU-bound DAX. Look at the query plan (Logical and Physical), find the expensive operator, refactor the measure (variables, replace iterators, avoid redundant context transitions).
 - High row counts on SE queries → scanning more than necessary; check that filters are pushed down.

I usually iterate measure-by-measure, re-running after each change, until total query time hits the target.

Q3. How do you use VertiPaq Analyzer (built into DAX Studio)?

Answer: VertiPaq Analyzer inspects the in-memory compressed model and shows:

- **Table size** — total size in memory.
- **Column size** — broken down by data and dictionary; identifies the heaviest columns.
- **Cardinality** — how many distinct values; high cardinality on a large column = poor compression.
- **Encoding** — value vs hash; hash means dictionary, often slower.
- **Relationships** — size and direction.

What I look for:

- High-cardinality columns I don't need (drop or split).
- DateTime columns that should be split into date + time (huge cardinality reduction).
- Free-text columns in fact tables that should be on dimensions.
- Unused columns from Power Query that snuck through.

Reducing model size typically improves performance — fewer bytes to scan.

Q4. What is TOAD and what do you use it for?

Answer: TOAD (Tool for Oracle Application Developers — now broader) is a developer/DBA toolset, most known for Oracle but with editions for SQL Server, DB2, MySQL, etc. Day-to-day uses:

- **Schema browsing and navigation** — tables, views, procs, dependencies.
- **SQL development** — editor with formatting, code completion, debugging.
- **Performance tuning** — explain plans, SQL optimizer, trace analysis, comparing execution plans.
- **Schema compare and synchronize** — diff between environments, generate migration scripts.
- **Data compare** — row-level differences between tables.
- **Automation** — scheduled scripts, code analysis, code review automation.

In a modern stack I use it primarily when working with legacy Oracle systems being migrated or integrated. For cloud-native DBs (Azure SQL, Synapse, Fabric Warehouse), I lean on SSMS, Azure Data Studio, or VS Code with database extensions.

Q5. How do you tune a slow SQL query using TOAD or any similar tool?

Answer: Standard playbook:

1. **Get the execution plan** — actual, not estimated. Look for full table scans, large hash joins, expensive sorts.
2. **Identify the bottleneck operator** — the one consuming the most rows or cost. Plans are read leaf-to-root in most cases.
3. **Check statistics** — stale stats produce bad plans. Refresh and re-plan.
4. **Look at predicates** — are filters sargable (index-usable) or wrapped in functions that prevent index use?
5. **Indexing** — missing indexes from the plan recommendations; check for unused indexes contributing to write cost.
6. **Join order and type** — sometimes a hint or rewrite (e.g., changing IN to EXISTS, eliminating correlated subqueries) helps.
7. **Data volume sanity** — is the query pulling more than it needs? Add filters, reduce columns.

8. **Parameter sniffing / plan stability** — for OLTP, watch for plans optimal for one parameter being awful for another.

I always measure twice — capture before/after with realistic data volumes, not against an empty dev DB.

Q6. How do you compare and migrate schemas across environments?

Answer: TOAD has Schema Compare; SQL Server has the same in SSDT/Azure Data Studio. The workflow:

1. Point at source (dev) and target (staging or prod).
2. Run compare; review object-by-object differences.
3. **Review carefully** — schema compare will happily generate drops for things that "shouldn't" be there. Always whitelist sensitive objects.
4. Generate migration script.
5. Test in a non-prod environment first.
6. Run with transaction scope where possible; have a rollback plan.

For real CI/CD: source-controlled schema (DACPACs for SQL Server, Liquibase or Flyway for cross-platform). Schema comparison is a debugging tool, not a release process — release should be deterministic from version-controlled definitions.

Appendix — Quick Reference Topics

A few cross-cutting things interviewers like to probe:

- **Cost-quality-latency triangle in GenAI** — be ready to discuss model selection (smaller for high-volume routing, larger for hard reasoning), caching strategies, prompt optimization, and batching.
- **Star schema design for Power BI** — single fact, conformed dimensions, no bidirectional relationships unless absolutely necessary, hide foreign keys.
- **Delta Lake essentials** — ACID, time travel, OPTIMIZE/VACUUM, MERGE, schema evolution flags.
- **Networking on Azure** — private endpoints, service endpoints, managed VNets, when each is appropriate.
- **Identity** — managed identities everywhere; service principals only when MI isn't possible; never account keys in code.

Good luck with the interviews — the strongest candidates pair concrete examples with their answers. Wherever possible, anchor each response to something you actually built, the trade-offs you considered, and how you measured success.